

Guía de Estudio – Sistema de Apuestas (MVC + Express + SQL Server)

1. Arquitectura General (MVC)

El proyecto sigue el patrón ****Modelo-Vista-Controlador (MVC)****:

...

```
ApuestasGPT/
├── App.js                → Punto de entrada (levanta el servidor)
├── Config/
│   └── db.js             → Conexión a la base de datos
├── Models/
│   └── Apuesta.js        → MODELO: lógica de negocio + consultas SQL
├── Controllers/
│   └── ApuestaController.js → CONTROLADOR: maneja req/res
├── Routes/
│   └── ApuestaRoutes.js  → RUTAS: define los endpoints
├── FrontEnd/
│   ├── index.html        → VISTA: listado de apuestas
│   ├── detalle.html      → VISTA: detalle de una apuesta
│   ├── main.js           → Lógica del listado (Frontend)
│   ├── detalle.js        → Lógica del detalle (Frontend)
│   ├── styles.css        → Estilos del listado
│   └── detalle.css       → Estilos del detalle
└── BaseDatos.sql         → Script de BD con tablas + datos
...
```

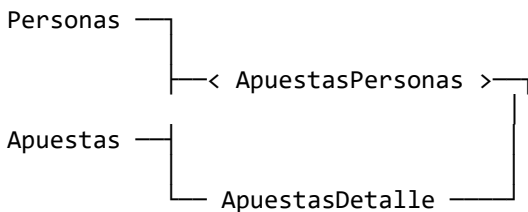
****Flujo de una petición:****

1. El navegador hace `GET http://localhost:3000/api/apuestas`
2. Express recibe la petición → la envía al ****Router****
3. El Router llama al ****Controlador**** (`obtenerApuestas`)
4. El Controlador llama al ****Modelo**** (`Apuesta.obtenerTodas()`)
5. El Modelo ejecuta la consulta SQL y devuelve los datos
6. Los datos vuelven al Controlador → responde con `res.json()`
7. El Frontend recibe la respuesta JSON y renderiza las cards

2. Base de Datos (SQL Server)

Diagrama de Tablas

...



...

Tabla: Personas

```
```sql
id_persona INT PK IDENTITY
apellido VARCHAR(50)
nombre VARCHAR(50)
dni VARCHAR(12) UNIQUE
fecha_nacimiento DATE
mail VARCHAR(100) UNIQUE
telefono VARCHAR(20)
saldo DECIMAL(12,2) DEFAULT 100000
estado CHAR(3) DEFAULT 'ACT' -- ACT=Activo, BAJ=Baja
```
```

Tabla: Apuestas

```

```sql
id_apuesta INT PK IDENTITY
evento VARCHAR(255)
fecha_evento DATETIME
fecha_cierre DATETIME
prioridad INT DEFAULT 0
comision DECIMAL(5,2) DEFAULT 0
estado CHAR(3) DEFAULT 'ACT' -- ACT=Abierta, CER=Cerrada, FIN=Finalizada,
BAJ=Baja
destacada BIT DEFAULT 0 -- 0=No, 1=Sí
```

```

Tabla: ApuestasDetalle

```

```sql
id_opcion INT PK IDENTITY
id_apuesta INT FK → Apuestas
num_ocurrencia INT -- 1=Opción A, 2=Opción B
descripcion VARCHAR(255) -- Ej: "Gana River Plate"
ocurrio CHAR(1) DEFAULT 'N' -- S=Sí, N=No
```

```

- Clave única compuesta: `(id\_apuesta, num\_ocurrencia)` – cada apuesta tiene exactamente 2 opciones

Tabla: ApuestasPersonas

```

```sql
id_jugada INT PK IDENTITY
id_apuesta INT FK
num_ocurrencia INT -- 1 o 2 (a qué opción apostó)
id_persona INT FK → Personas
fecha DATETIME DEFAULT CURRENT_TIMESTAMP
importe DECIMAL(12,2) -- >0 y ≤100000
```

```

- FK compuesta hacia `ApuestasDetalle(id\_apuesta, num\_ocurrencia)`
- Cada registro representa una ****apuesta realizada por una persona****

Estados posibles

| Código | Significado |
|--------|--------------------------------------|
| ACT | Abierta (se puede apostar) |
| CER | Cerrada (no se aceptan más apuestas) |
| FIN | Finalizada (el evento ocurrió) |
| BAJ | Baja (eliminada lógicamente) |

3. Backend – Explicación por archivos

3.1. `App.js` (Servidor Express)

```

```javascript
const express = require("express");
const cors = require("cors");
const apuestaRoutes = require("./Routes/ApuestaRoutes");

const app = express();
app.use(cors()); // Permite peticiones desde otros orígenes
app.use(express.json()); // Interpreta cuerpos JSON
app.use(express.static("FrontEnd")); // Sirve archivos estáticos (HTML, CSS, JS)
app.use("/api/apuestas", apuestaRoutes); // Monta las rutas bajo /api/apuestas

app.listen(3000, () => {
 console.log("Servidor en puerto 3000");
});
```

```

- ****CORS****: Necesario porque el frontend corre en el mismo origen pero el backend está en otro puerto/lugar
- ****express.static****: Hace que `localhost:3000` sirva `FrontEnd/index.html` automáticamente
- ****app.use("/api/apuestas", ...)****: Todas las rutas de apuestas arrancan con `/api/apuestas`

3.2. `Config/db.js` (Conexión a SQL Server)

```
```javascript
const sql = require("mssql/msnodesqlv8");

const stringDeConexion = "Driver={ODBC Driver 17 for SQL Server};"
 + "Server=(local)\\SQLEXPRESS;Database=SistemasApuesta;Trusted_Connection=yes;";

async function getConnection() {
 const pool = await sql.connect({ connectionString: stringDeConexion });
 return pool;
}
```
```

- Usa **Windows Authentication** (`Trusted_Connection=yes`)
- Devuelve un **pool de conexiones** (reutilizable, no abre una conexión nueva cada vez)
- Exporta `sql` para usar los tipos (ej: `sql.Int`, `sql.Char`)

3.3. `Routes/ApuestaRoutes.js` (Definición de rutas)

```
```javascript
router.get("/", apuestaController.obtenerApuestas);
router.get("/:id", apuestaController.obtenerApuestaPorId);
router.get("/:id/apostadores", apuestaController.obtenerApostadores);
router.put("/:id/estado", apuestaController.cambiarEstado);
router.put("/:id/destacar", apuestaController.destacarApuesta);
```
```

| Método | Endpoint | Función |
|--------|--|------------------------------------|
| GET | <code>/api/apuestas</code> | Listar todas las apuestas |
| GET | <code>/api/apuestas/:id</code> | Obtener una apuesta por ID |
| GET | <code>/api/apuestas/:id/apostadores</code> | Obtener apostadores de una apuesta |
| PUT | <code>/api/apuestas/:id/estado</code> | Abrir o cerrar una apuesta |
| PUT | <code>/api/apuestas/:id/destacar</code> | Activar/desactivar destacada |

3.4. `Controllers/ApuestaController.js` (Manejo de req/res)

El controlador recibe `req` y `res`, llama al modelo, y responde.

estructura típica de cada función:

```
```javascript
async function obtenerApuestas(req, res) {
 try {
 const apuestas = await Apuesta.obtenerTodas();
 res.status(200).json(apuestas);
 } catch (error) {
 console.log(error);
 res.status(500).json({ mensaje: "Error al obtener las apuestas" });
 }
}
```
```

- **try/catch**: Manejo de errores. Si falla la BD, responde 500
- **res.status(200).json()**: Respuesta exitosa con datos
- **res.status(404).json()**: Recurso no encontrado
- **res.status(500).json()**: Error del servidor

Funciones del controlador:

| Función | Parámetros | Respuesta |
|----------------------------------|----------------------------|-----------------------------|
| <code>obtenerApuestas</code> | - | Array de todas las apuestas |
| <code>obtenerApuestaPorId</code> | <code>req.params.id</code> | Una apuesta o 404 |
| <code>obtenerApostadores</code> | <code>req.params.id</code> | Array de apostadores |
| <code>cambiarEstado</code> | <code>req.params.id</code> | Mensaje + nuevo estado |
| <code>destacarApuesta</code> | <code>req.params.id</code> | Mensaje de confirmación |

3.5. `Models/Apuesta.js` (Lógica de negocio y consultas SQL)

`obtenerTodas()` – La función más importante

```
````javascript
async function obtenerTodas() {
 const pool = await getConnection();
 const resultado = await pool.request().query(`...`);
 const apuestas = [];
 // 1. AGRUPAR: Itera el resultado y agrupa por id_apuesta
 for (let fila of resultado.recordset) {
 let apuestaExistente = apuestas.find(a => a.id_apuesta === fila.id_apuesta);
 if (!apuestaExistente) {
 apuestaExistente = { ... };
 apuestas.push(apuestaExistente);
 }
 // 2. ACUMULAR: Suma los montos apostados a cada opción
 if (fila.num_ocurrencia === 1) apuestaExistente.montoOpcionA += fila.importe;
 if (fila.num_ocurrencia === 2) apuestaExistente.montoOpcionB += fila.importe;
 }
 // 3. CALCULAR CUOTAS
 apuestas.forEach(apuesta => {
 const totalApostado = apuesta.montoOpcionA + apuesta.montoOpcionB;
 const pozoNeto = totalApostado - (totalApostado * apuesta.comision / 100);
 apuesta.cuotaA = pozoNeto / apuesta.montoOpcionA;
 apuesta.cuotaB = pozoNeto / apuesta.montoOpcionB;
 });
 return apuestas;
}
````

**La consulta SQL:**
````sql
SELECT A.id_apuesta, A.evento, A.fecha_evento, A.fecha_cierre,
 A.prioridad, A.destacada, A.comision, A.estado,
 O1.descripcion AS opcionA, O2.descripcion AS opcionB,
 AP.num_ocurrencia, AP.importe
FROM Apuestas A
INNER JOIN ApuestasDetalle O1 ON A.id_apuesta = O1.id_apuesta AND O1.num_ocurrencia = 1
INNER JOIN ApuestasDetalle O2 ON A.id_apuesta = O2.id_apuesta AND O2.num_ocurrencia = 2
LEFT JOIN ApuestasPersonas AP ON A.id_apuesta = AP.id_apuesta
ORDER BY
 CASE WHEN A.destacada = 1 THEN 0
 WHEN A.fecha_cierre >= GETDATE() AND A.estado != 'CER' THEN 1
 WHEN A.fecha_cierre < GETDATE() AND A.estado != 'CER' THEN 2
 ELSE 3 END,
 A.fecha_cierre ASC
````

**Explicación del JOIN:**
- `INNER JOIN` con `ApuestasDetalle` dos veces: una para la opción A (ocurrencia 1) y otra para la opción B (ocurrencia 2)
- `LEFT JOIN` con `ApuestasPersonas`: puede haber 0, 1 o N personas que apostaron

**ORDER BY explicado (prioridad de ordenamiento):**
1. **Destacadas primero** (`CASE WHEN destacada = 1 THEN 0`)
2. **No vencidas no cerradas** (`fecha_cierre >= GETDATE() AND estado != 'CER'`)
3. **Vencidas no cerradas** (`fecha_cierre < GETDATE() AND estado != 'CER'`)
4. **Cerradas** (ELSE)
5. Dentro de cada grupo, por **fecha_cierre ascendente** (más próxima primero)

**Cálculo de cuotas:**
````
Total Apostado = MontoOpciónA + MontoOpciónB
Pozo Neto = TotalApostado - (TotalApostado * Comisión / 100)
Cuota A = PozoNeto / MontoOpciónA (Si MontoOpciónA > 0, sino 0)
Cuota B = PozoNeto / MontoOpciónB (Si MontoOpciónB > 0, sino 0)
````
```

La cuota representa ****lo que ganaría quien apostó \$1**** si su opción resulta ganadora. A más gente apueste a una opción, menor será su cuota (menor ganancia potencial).

`obtenerPorId(id)` – Una sola apuesta con sus montos

Misma lógica que `obtenerTodas()` pero filtrada por `WHERE A.id\_apuesta = @id`. Retorna un solo objeto o `null`.

`cambiarEstado(id)` – Toggle entre ACT y CER

```
```javascript
const estadoActual = consulta.recordset[0].estado;
const nuevoEstado = estadoActual === "ACT" ? "CER" : "ACT";
// UPDATE Apuestas SET estado = @estado WHERE id_apuesta = @id
```
```

- Lee el estado actual de la BD
- Si está ACT → lo pasa a CER (cerrar)
- Si está CER → lo pasa a ACT (reabrir)

`destacarApuesta(id)` – Toggle del bit destacada

```
```javascript
UPDATE Apuestas SET destacada = CASE WHEN destacada = 1 THEN 0 ELSE 1 END
```
```

Usa un `CASE` en SQL para invertir el valor: si es 1 pasa a 0, si es 0 pasa a 1.

`obtenerApostadores(id)` – Personas que apostaron

```
```sql
SELECT P.nombre, P.apellido, AP.importe, D.descripcion AS opcion
FROM ApuestasPersonas AP
INNER JOIN Personas P ON AP.id_persona = P.id_persona
INNER JOIN ApuestasDetalle D ON AP.id_apuesta = D.id_apuesta AND AP.num_ocurrencia =
D.num_ocurrencia
WHERE AP.id_apuesta = @id
ORDER BY P.apellido, P.nombre
```
```

Retorna un array con: nombre, apellido, importe apostado, y qué opción eligió.

4. Frontend – Explicación por archivos

4.1. `index.html` (Listado de apuestas)

```
```html
<header class="header">
 <h1>Panel Administrador - Sistema de Apuestas</h1>
</header>
<main class="contenido">
 <section id="mensajeError" class="mensaje-error mensaje-error--oculto">
 No se pudieron cargar las apuestas.
 </section>
 <section id="contenedorApuestas" class="apuestas">
 <!-- Las cards se generan dinámicamente con JS -->
 </section>
</main>
<footer class="footer">
 <p>Tecnica en Programación - Sistema de Apuestas</p>
</footer>
<script src="main.js"></script>
```
```

- `#contenedorApuestas`: contenedor vacío que se llena con JS
- `#mensajeError`: oculto por defecto, se muestra si falla el fetch
- Usa BEM (Bloque Elemento Modificador) para las clases CSS

4.2. `main.js` (Lógica del listado)

Flujo de arranque:

```
```javascript
document.addEventListener("DOMContentLoaded", cargarApuestas);
```
```

apenas se carga el DOM, llama a `cargarApuestas()`.

`cargarApuestas()` – async/await

```
```javascript
async function cargarApuestas() {
 const respuesta = await fetch(URL_API); // GET a /api/apuestas
 const apuestas = await respuesta.json(); // Parsea el JSON
 contenedorApuestas.innerHTML = ""; // Limpia el contenedor
 apuestas.forEach(apuesta => {
 const card = crearCard(apuesta); // Crea una card
 contenedorApuestas.appendChild(card); // La agrega al DOM
 });
}
```
```

****Uso de fetch con async/await**:** Espera a que el servidor responda sin bloquear el navegador.

`crearCard(apuesta)` – DOM puro, sin frameworks

Crea cada elemento HTML con `document.createElement()` y los ensambla.

****Clases dinámicas:****

```
```javascript
card.classList.add("card");
if (apuesta.destacada) card.classList.add("card--destacada");
else if (apuesta.estado === "CER") card.classList.add("card--cerrada");
// Estado: clase verde si ACT, roja si CER
if (apuesta.estado === "ACT") {
 estado.classList.add("card__estado--abierta");
 estado.textContent = "Estado: Abierta";
} else {
 estado.classList.add("card__estado--cerrada");
 estado.textContent = "Estado: Cerrada";
}
```
```

****Botones condicionales (solo aparecen si la apuesta NO está cerrada):****

```
```javascript
if (apuesta.estado !== "CER") {
 // Botón Destacar
 // Botón Apostar (disabled = true, solo visual)
}
```
```

- ****Botón Abrir/Cerrar**:** Siempre visible. Cambia de clase y texto según estado
- ****Botón Destacar**:** Solo en apuestas no cerradas
- ****Botón Apostar**:** Solo en no cerradas (pero deshabilitado – es solo interfaz admin)
- ****Botón Ver Detalle**:** Siempre visible. Redirige a `detalle.html?id=...`

****Eventos:****

```
```javascript
botonEstado.addEventListener("click", () => cambiarEstado(apuesta.id_apuesta));
botonDestacar.addEventListener("click", () => destacarApuesta(apuesta.id_apuesta));
botonDetalle.addEventListener("click", () => { window.location.href = "detalle.html?id=..."; });
```
```

`cambiarEstado(id)` y `destacarApuesta(id)`

```
```javascript
async function cambiarEstado(id) {
 const respuesta = await fetch(`${URL_API}/${id}/estado`, { method: "PUT" });
}
```

```

 if (!respuesta.ok) throw new Error("Error");
 cargarApuestas(); // Recarga la lista completa actualizada
}
...

```

- Hacen un `fetch PUT` al endpoint correspondiente
- Si la respuesta es OK, \*\*recargan la lista\*\* (`cargarApuestas()`) para reflejar los cambios
- Si falla, muestran `alert()` con el error

```

`formatearFecha(fecha)`
```javascript
function formatearFecha(fecha) {
    return new Date(fecha).toLocaleString("es-AR");
}
...

```

Convierte la fecha ISO de SQL Server a formato legible argentino (dd/mm/aaaa hh:mm:ss).

4.3. `detalle.html` (Página de detalle)

```

```html
<section id="detalleApuesta" class="detalle">
 <!-- Contenido generado por JS -->
</section>
<section class="apostadores">
 <h2>Personas que apostaron</h2>
 <table class="tabla">
 <thead>
 <tr><th>Persona</th><th>Opción</th><th>Importe</th></tr>
 </thead>
 <tbody id="tablaApostadores">
 <!-- Filas generadas por JS -->
 </tbody>
 </table>
</section>
...

```

### ### 4.4. `detalle.js` (Lógica del detalle)

```

Obtener ID de la URL:
```javascript
const parametros = new URLSearchParams(window.location.search);
const idApuesta = parametros.get("id");
...

```

Toma el `?id=5` de la URL usando `URLSearchParams`.

`cargarDetalle()` y `mostrarDetalle(apuesta)`

```

```javascript
async function cargarDetalle() {
 const respuesta = await fetch(`${URL_API}/${idApuesta}`);
 const apuesta = await respuesta.json();
 mostrarDetalle(apuesta);
}
...

```

`mostrarDetalle()` crea elementos DOM para:

- Título (evento)
- Fecha evento, fecha cierre, estado
- Opción A con monto y cuota
- Opción B con monto y cuota
- Botones: Abrir/Cerrar, Destacar (solo si no está CER), Volver

### #### `cargarApostadores()`

```

```javascript
async function cargarApostadores() {
    const respuesta = await fetch(`${URL_API}/${idApuesta}/apostadores`);
    const apostadores = await respuesta.json();
    apostadores.forEach(apostador => {

```

```

        // Crea una fila <tr> con nombre, opción, importe
        tablaApostadores.appendChild(fila);
    });
}
...

```

Recorre el array y por cada apostador crea una fila de tabla con tres columnas.

`cambiarEstado()` y `destacarApuesta()`

```

```javascript
async function cambiarEstado() {
 await fetch(`${URL_API}/${idApuesta}/estado`, { method: "PUT" });
 location.reload(); // Recarga toda la página para reflejar cambios
}
...

```

A diferencia de `main.js` que recarga solo la lista, aquí se usa `location.reload()` porque la página completa debe actualizarse.

### 4.5. `styles.css` y `detalle.css` (Estilos)

**\*\*Reset básico:\*\***

```

```css
* { margin: 0; padding: 0; box-sizing: border-box; }
...

```

****Cards (listado):****

```

```css
.card {
 width: 400px;
 background: white;
 border-radius: 8px;
 padding: 20px;
 box-shadow: 0 2px 5px rgba(0,0,0,0.1);
 display: flex;
 flex-direction: column; /* Para que las acciones se vayan al fondo */
}

```

```

.card > :last-child { margin-top: auto; } /* Empuja acciones al fondo */

```

```

.card--destacada {
 border-left: 4px solid orange;
 background-color: #fff8e6;
}

```

```

.card--cerrada {
 border-left: 4px solid #c0392b;
 background-color: #fde6e6;
}
...

```

**\*\*Botones:\*\***

```

```css
.boton {
    padding: 10px 15px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    flex: 1; /* Todos los botones mismo ancho */
    min-width: 80px;
    height: 40px; /* Todos los botones mismo alto */
    display: flex;
    align-items: center;
    justify-content: center;
}

```

```

.boton--abrir { background: green; color: white; }
.boton--cerrar { background: crimson; color: white; }

```



```
.boton--destacar { background: orange; color: white; }
.boton--detalle  { background: steelblue; color: white; }
.boton--apostar  { background: rgb(81, 221, 81); color: white; }
...

```

****Contenedor de cards (grid flex):****

```
```css
.apuestas {
 display: flex;
 justify-content: center;
 flex-wrap: wrap;
 gap: 20px;
}
```

```

5. Preguntas Frecuentes para la Defensa

¿Por qué usaste SQL Server y no MySQL?

SQL Server se eligió porque la materia/entorno de estudio así lo requiere. Usamos `msnodesqlv8` para conectarnos con autenticación de Windows.

¿Qué es MVC y cómo se aplica acá?

MVC separa la aplicación en tres capas:

- ****Modelo**** (`Models/Apuesta.js`): datos y lógica de negocio (consultas SQL, cálculos)
- ****Vista**** (`FrontEnd/\*.html`, `\*.css`): interfaz de usuario
- ****Controlador**** (`Controllers/ApuestaController.js`): intermediario que recibe peticiones y orquesta la respuesta

¿Cómo funciona el cálculo de cuotas?

...

$$\text{Cuota} = \text{PozoNeto} / \text{MontoApostadoAOpcion}$$

...

Donde $\text{PozoNeto} = \text{TotalApostado} - (\text{TotalApostado} * \text{Comisión} / 100)$. Si todos apuestan a una opción, esa opción tiene cuota baja (poco retorno). La comisión es la ganancia de la casa.

¿Por qué el ORDER BY está en SQL y no en Node.js?

Originalmente estaba en Node.js con `.sort()`, pero se movió a SQL Server usando `CASE WHEN` en el `ORDER BY` para:

- Mejor performance (el motor de BD ordena más rápido)
- Menos código en Node.js
- La BD ya tiene los índices para ordenar

¿Cómo funciona el toggle de estado (ACT ↔ CER)?

El endpoint PUT recibe el ID, el modelo lee el estado actual de la BD, y si es ACT lo cambia a CER, y viceversa. Esto se llama "toggle" o "conmutación".

¿Qué hace `express.static`?

Hace que Express sirva archivos estáticos (HTML, CSS, JS, imágenes) sin necesidad de definir rutas específicas. Así `localhost:3000` sirve `FrontEnd/index.html` automáticamente.

¿Cómo maneja el frontend los errores del backend?

Con bloques `try/catch`:

- Si `fetch` falla (red), entra al catch
- Si `respuesta.ok` es false (código HTTP 4xx/5xx), lanza un error explícito
- En el listado: muestra un mensaje de error en pantalla
- En el detalle: muestra un `alert()`

Explica la relación entre las tablas

- **Apuestas** 1 → N **ApuestasDetalle**: Cada apuesta tiene 2 opciones (ocurrencia 1 y 2)
- **Apuestas** 1 → N **ApuestasPersonas**: Cada apuesta puede tener muchas personas que apostaron
- **Personas** 1 → N **ApuestasPersonas**: Cada persona puede apostar en múltiples apuestas
- **ApuestasDetalle** 1 → N **ApuestasPersonas**: Cada opción puede tener muchas apuestas de personas

¿Por qué el LEFT JOIN en la consulta principal?

Porque una apuesta puede no tener aún personas que hayan apostado. Con `INNER JOIN` se perderían esas apuestas. Con `LEFT JOIN`, si no hay apostadores, `AP.importe` es `NULL` y se maneja con `if (fila.importe)`.

¿Qué es `IDENTITY(1,1)` en SQL Server?

Es un campo auto-incremental. Empieza en 1 y aumenta de 1 en 1. SQL Server asigna automáticamente el valor, no hace falta insertarlo manualmente.

¿Qué es `CURRENT\_TIMESTAMP`?

Es una función de SQL Server que devuelve la fecha y hora actual del servidor. Se usa como valor por defecto en `ApuestasPersonas.fecha`.

6. Cómo ejecutar el proyecto

```
```bash
```

```
1. Ejecutar BaseDatos.sql en SQL Server Management Studio (SSMS)
```

```
2. Instalar dependencias
```

```
cd ApuestasGPT
```

```
npm install
```

```
3. Iniciar servidor (con nodemon para desarrollo)
```

```
npm run dev
```

```
4. Abrir navegador en
```

```
http://localhost:3000
```

```
```
```

Scripts disponibles:

| Comando | Descripción |
|---------------|---|
| `npm run dev` | Inicia con nodemon (recarga automática al cambiar código) |
| `npm start` | Inicia con node (sin recarga automática) |

7. Endpoints de la API

| Método | Ruta | Cuerpo | Respuesta |
|--------|---------------------------------|--------|---|
| GET | `/api/apuestas` | - | `[{ id_apuesta, evento, ... }]` |
| GET | `/api/apuestas/:id` | - | `{ id_apuesta, evento, ... }` |
| GET | `/api/apuestas/:id/apostadores` | - | `[{ nombre, apellido, importe, opcion }]` |
| PUT | `/api/apuestas/:id/estado` | - | `{ mensaje, estado }` |
| PUT | `/api/apuestas/:id/destacar` | - | `{ mensaje }` |

8. Buenas prácticas aplicadas

1. **Async/Await**: Toda operación asíncrona (fetch, BD) usa async/await
2. **Manejo de errores**: try/catch en cada función que hace llamadas externas
3. **DOM puro**: Sin frameworks JS, usando createElement y classList
4. **BEM**: Nomenclatura de clases CSS (`card\_titulo`, `card--destacada`)
5. **Parámetros**: Consultas SQL parametrizadas (.input()) para evitar SQL injection
6. **Separación de responsabilidades**: MVC bien definido
7. **Comisiones**: Modeladas como porcentaje descontado del pozo total

8. ****Flexbox****: Cards y botones con diseño flexible y simétrico
9. ****No hardcodeo****: URL de API definida como constante, fácil de cambiar
10. ****Estados con CHECK****: SQL valida que los estados sean válidos